EE709 Assignment 1

Autumn Semester 2026, IIT Bombay

Submitted by: Kusumuru Suresh Kumar (26M2029)

https://github.com/suresh-vlsi/EE709_Assignment1

Problem statement:

You are given a sequential circuit with inputs $reset, r0, r1$, state variables $q0, q1, p$ and outputs $g0, g1$, described by the following equations:

$$q_0(k+1) = \overline{reset(k)} \cdot \left(r_0(k) \cdot \overline{q_0(k)} + \overline{g_0(k)} \cdot q_0(k) \right)$$

$$q_1(k+1) = \overline{reset(k)} \cdot \left(r_1(k) \cdot \overline{q_1(k)} + \overline{g_1(k)} \cdot q_1(k) \right)$$

$$p(k+1) = \overline{reset(k)} \cdot \left(p(k) \oplus \left(g_0(k) + g_1(k) \right) \right)$$

$$g_0(k) = \left(p(k) \ ? \ q_0(k) \cdot \overline{q_1(k)} : \ q_0(k) \right)$$

$$g_1(k) = \left(p(k) \ ? \ q_1(k) : \ q_1(k) \cdot \overline{q_0(k)} \right)$$

1. **Reachable set using BDD's**:

Determine, using ROBDD's. the set of reachable states $(p, q0, q1)$assuming that $reset(0) = 1$, and $reset(k) = 0$for all k > 0. The other inputs are not constrained in any way. Enumerate the states that can be reached.

2. **Property checking using SAT**:

1. Determine, using SAT, if the following properties hold
   a) $\mathrm{AG}(r_0 \Rightarrow \mathrm{F} g_0)$
   b) $\mathrm{AG}(r_1 \Rightarrow \mathrm{F} g_1)$
   c) $\neg \mathrm{EF}(g_0 \cdot g_1)$

   Note: establish your proof over a window of eight cycles.
2. Now assume that the circuit is modified by making $p(k) = 0 \ \forall k$. Determine, using SAT, if the property $\mathrm{AG}(r_1 \Rightarrow \mathrm{F} g_1)$ still holds. Note: establish your proof over a window of eight cycles.

Approach:

Notation: ! for NOT, · for AND, + for OR, ⊕ for XOR. Therefore, for k ≥ 1 (reset = 0):

- q0+ = r0·!q0 + q0·!g0
- q1+ = r1·!q1 + q1·!g1
- p+ = p ⊕ (g0+g1)
- g0 = p ? q0·!q1 : q0
- g1 = p ? q1 : q1·!q0

Initially, reset(0) = 1 forces the state at cycle 1 to (p,q0,q1) = 000 whatever the state or inputs at cycle 0 are. Hence R0 = {000}.

Approach to problems:

1. Question 1 Symbolic reachability:
   - Build T(s,i,s') = ∧ (x' ≡ fx(s,i)) with reset = 0;
   - iterate Rn+1 = Rn + subst(∃(p,q0,q1,r0,r1)·Rn·T, s' → s) until Rn+1 = Rn (an identity test, since ROBDDs are canonical).

   Solved by hand, then confirmed by program.
2. Question 2 Bounded model checking over a window of eight cycles (k = 0..7): the circuit is unrolled, converted to CNF with Tseitin gates, the negation of the property is asserted, and a SAT solver (MiniSat22 through PySAT) is queried. UNSAT means no violation in the window. Each result is also proved by hand.

Evidence:
1. Hand derivations, 2. program listings, 3. terminal logs, and 4. the commands and software needed to reproduce everything.

Question 1:
Solution:

With reset=0 after cycle 0, the equations are:
q0+ = r0·!q0 + q0·!g0
q1+ = r1·!q1 + q1·!g1
p+ = p ⊕ (g0+g1)
g0 = p ? q0·!q1 : q0
g1 = p ? q1 : q1·!q0
Since reset(0)=1, the state after cycle 0 is 000, so R_0 = {000}.

Step 1: grants in each state

State (p q0 q1)	g0	g1	Any grant?
000	0	0	no
001	0	1	yes
010	1	0	yes
011	1	0	yes
100	0	0	no
101	0	1	yes
110	1	0	yes
111	0	1	yes

For p=0, g0=q0 and g1=q1·!q0. For p=1, g0=q0·!q1 and g1=q1.

Step 2: next-state table
If q=0, then q+ = r.

If q=1, then q+ = !g, because r·!q is 0.
p toggles when any grant occurs.

State	p+	q0+	q1+	Successors (over r0, r1)
000	0	r0	r1	000, 001, 010, 011
001	1	r0	0	100, 110
010	1	0	r1	100, 101
011	1	0	1	101
100	1	r0	r1	100, 101, 110, 111
101	0	r0	0	000, 010
110	0	0	r1	000, 001
111	0	1	0	010

Step 3: fixpoint iteration
$R_{n+1} = R_n \cup \text{Image}(R_n)$.

Iteration	New states	R_n	ROBDD
R_0	000	{000}	!p·!q0·!q1 (3 nodes)
R_1	001, 010, 011 (from 000)	all four p=0 states	!p (1 node)
R_2	100, 110 (from 001); 100, 101 (from 010); 101 (from 011)	7 states, all except 111	!(p·q0·q1) (3 nodes)
R_3	111 (from 100 with r0=r1=1)	all 8 states	constant 1 (0 nodes)
R_4	none	$R_4 = R_3$	fixpoint

Therefore, all 8 states are reachable, and the fixpoint is reached at iteration 4.

Implementation:
1. bdd_reach.py:

```
#!/usr/bin/env python3
"""EE709 Assignment 1, Part 1: reachable states of the arbiter using ROBDDs.
Self-contained ROBDD package (unique table + ITE + exists + rename), no external
library.
Run: python3 bdd_reach.py | tee bdd_run.log

Equations (overbars restored), reset(0)=1 and reset(k>0)=0:
  q0+ = !reset.(r0.!q0 + q0.!g0)   q1+ = !reset.(r1.!q1 + q1.!g1)   p+ =
!reset.(p xor (g0+g1))
  g0 = p ? q0.!q1 : q0         g1 = p ? q1 : q1.!q0
"""
import itertools

class BDD:
    def __init__(s):
        s.node = [None, None]          # node id -> (var, lo, hi); ids 0,1 are
the constants
        s.uniq, s.memo = {}, {}
    def mk(s, v, lo, hi):
        if lo == hi: return lo         # redundant test removed (reduced)
        k = (v, lo, hi)
        if k not in s.uniq:            # shared nodes (ordered + reduced =
canonical)
```

```
            s.node.append(k); s.uniq[k] = len(s.node) - 1
        return s.uniq[k]
    def var(s, i): return s.mk(i, 0, 1)
    def lvl(s, u): return 10**9 if u < 2 else s.node[u][0]
    def cof(s, u, v):
        if u < 2 or s.node[u][0] != v: return u, u
        return s.node[u][1], s.node[u][2]
    def ite(s, f, g, h):
        if f == 1: return g
        if f == 0: return h
        if g == h: return g
        if g == 1 and h == 0: return f
        k = ('ite', f, g, h)
        if k in s.memo: return s.memo[k]
        v = min(s.lvl(f), s.lvl(g), s.lvl(h))
        (f0, f1), (g0, g1), (h0, h1) = s.cof(f, v), s.cof(g, v), s.cof(h, v)
        r = s.mk(v, s.ite(f0, g0, h0), s.ite(f1, g1, h1))
        s.memo[k] = r; return r
    def NOT(s, f): return s.ite(f, 0, 1)
    def AND(s, f, g): return s.ite(f, g, 0)
    def OR(s, f, g): return s.ite(f, 1, g)
    def XOR(s, f, g): return s.ite(f, s.NOT(g), g)
    def XNOR(s, f, g): return s.ite(f, g, s.NOT(g))
    def exists(s, f, vs):
        if f < 2: return f
        k = ('ex', f, frozenset(vs))
        if k in s.memo: return s.memo[k]
        v, lo, hi = s.node[f]
        a, b = s.exists(lo, vs), s.exists(hi, vs)
        r = s.OR(a, b) if v in vs else s.mk(v, a, b)
        s.memo[k] = r; return r
    def substitute(s, f, m):           # replace variable i by variable m[i]
        if f < 2: return f
        k = ('sub', f, tuple(sorted(m.items())))
        if k in s.memo: return s.memo[k]
        v, lo, hi = s.node[f]
        r = s.ite(s.var(m.get(v, v)), s.substitute(hi, m), s.substitute(lo, m))
        s.memo[k] = r; return r
    def size(s, f):
        seen, st = set(), [f]
        while st:
            u = st.pop()
            if u > 1 and u not in seen: seen.add(u); st += [s.node[u][1],
s.node[u][2]]
        return len(seen)
    def eval(s, f, a):                 # a: var index -> 0/1
        while f > 1: v, lo, hi = s.node[f]; f = hi if a[v] else lo
        return f

B = BDD()
# variable order = creation order: present state, next state, inputs
```

```
P, Q0, Q1, Pn, Q0n, Q1n, R0, R1 = range(8)
p, q0, q1, pn, q0n, q1n, r0, r1 = (B.var(i) for i in range(8))

g0 = B.ite(p, B.AND(q0, B.NOT(q1)), q0)
g1 = B.ite(p, q1, B.AND(q1, B.NOT(q0)))
fq0 = B.OR(B.AND(r0, B.NOT(q0)), B.AND(q0, B.NOT(g0)))          # reset = 0
fq1 = B.OR(B.AND(r1, B.NOT(q1)), B.AND(q1, B.NOT(g1)))
fp  = B.XOR(p, B.OR(g0, g1))
T = B.AND(B.AND(B.XNOR(pn, fp), B.XNOR(q0n, fq0)), B.XNOR(q1n, fq1))
print(f"transition relation T: {B.size(T)} BDD nodes")

# reset(0)=1 forces the state after cycle 0 to (p,q0,q1)=(0,0,0)
R = B.AND(B.AND(B.NOT(p), B.NOT(q0)), B.NOT(q1))
quant = {P, Q0, Q1, R0, R1}                   # quantify present state and
inputs
back = {Pn: P, Q0n: Q0, Q1n: Q1}              # next-state vars -> present-state
vars

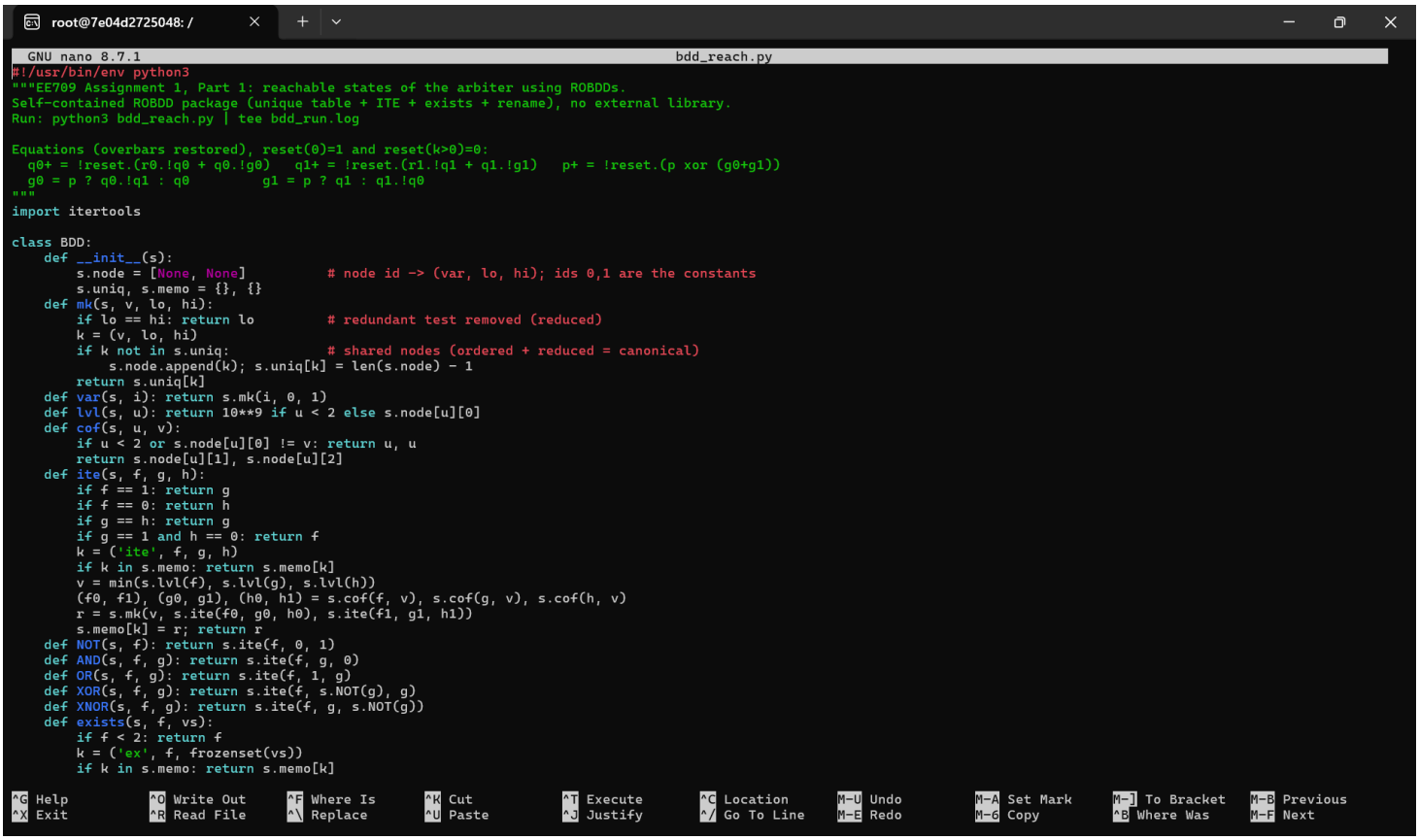
def states(f):
    return [(a, b, c) for a, b, c in itertools.product((0, 1), repeat=3)
            if B.eval(f, {P: a, Q0: b, Q1: c})]
it = 0
print(f"R{it}: {len(states(R))} states, {B.size(R)} nodes, {states(R)}")
while True:
    img = B.substitute(B.exists(B.AND(R, T), quant), back)
    Rn = B.OR(R, img); it += 1
    print(f"R{it}: {len(states(Rn))} states, {B.size(Rn)} nodes, {states(Rn)}")
    if Rn == R: break                         # canonical => pointer/id equality
test
    R = Rn
print("\nFixpoint reached at iteration", it)
print("R is constant 1 (all 8 states reachable):", R == 1)
print("Reachable (p,q0,q1):", [f"{a}{b}{c}" for a, b, c in states(R)])
```

2. expected run log:

```
transition relation T: 35 BDD nodes
R0: 1 states, 3 nodes, [(0, 0, 0)]
R1: 4 states, 1 nodes, [(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1)]
R2: 7 states, 3 nodes, [(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0),
(1, 0, 1), (1, 1, 0)]
R3: 8 states, 0 nodes, [(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0),
(1, 0, 1), (1, 1, 0), (1, 1, 1)]
R4: 8 states, 0 nodes, [(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0),
(1, 0, 1), (1, 1, 0), (1, 1, 1)]

Fixpoint reached at iteration 4
R is constant 1 (all 8 states reachable): True
Reachable (p,q0,q1): ['000', '001', '010', '011', '100', '101', '110', '111']
```

3. insert code in nano:



4. Output terminal:

```
(sat-env) root@7e04d2725048:/# nano bdd_reach.py
(sat-env) root@7e04d2725048:/#    cd /
   python3 bdd_reach.py | tee bdd_run.log
transition relation T: 35 BDD nodes
R0: 1 states, 3 nodes, [(0, 0, 0)]
R1: 4 states, 1 nodes, [(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1)]
R2: 7 states, 3 nodes, [(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0), (1, 0, 1), (1, 1, 0)]
R3: 8 states, 0 nodes, [(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 1, 1)]
R4: 8 states, 0 nodes, [(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 1, 1)]

Fixpoint reached at iteration 4
R is constant 1 (all 8 states reachable): True
Reachable (p,q0,q1): ['000', '001', '010', '011', '100', '101', '110', '111']
(sat-env) root@7e04d2725048:/#
```

Figure 1: Run log of bdd_reach.py. T has 35 nodes; per-iteration states and node counts agree with the hand table; R is the constant-1 ROBDD.

Question 2:
Solution:

For k ≥ 1, reset=0 and the equations are:

- q0+ = r0·!q0 + q0·!g0
- q1+ = r1·!q1 + q1·!g1
- p+ = p ⊕ (g0+g1)
- g0 = p ? q0·!q1 : q0
- g1 = p ? q1 : q1·!q0

2.1 (c): **¬EF(g0·g1)**

- If p=1: g0·g1 = (q0·!q1)·q1 = 0.
- If p=0: g0·g1 = q0·(q1·!q0) = 0.

So g0·g1 is 0 in every state at every cycle, and the property holds. This does not depend on the window or on reachability.

2.1 (a): **AG(r0 ⇒ F g0)**

- q0 is held until it is granted. If q0=1 and g0=0, then q0+ = q0·!g0 = 1.
- if q0(k)=1, then g0 fires at k or k+1.
  i. If p=0, then g0=q0=1 immediately.
  ii. If p=1 and q1=0, then g0=q0=1 immediately.
  iii. If p=1 and q1=1, then g0=0 and g1=1. So p+ = 1⊕1 = 0, and q0+ = q0·!g0 = 1. At k+1, p=0 and q0=1, so g0=1.

Request at cycle k ≥ 1 with r0(k)=1.

- If q0(k)=0, then q0(k+1)=r0·!q0=1. By Fact B, g0 fires at k+1 or k+2.
- If q0(k)=1, then by Fact B, g0 fires at k or k+1.

Either way, g0 fires within k..k+2. A violation would need ¬g0 on all of k..7, which is impossible when k+2 ≤ 7, i.e. k = 1..5. So the SAT query for those k is UNSAT, and the property holds.
The three special cases:

k	Result	Why
0	SAT	reset(0)=1 gives q0(1)=0, so the request is discarded. Take r0(0)=1, q0(0)=0, r0=0 afterwards: q0 stays 0 and g0 never fires.
6	SAT	The grant needs k+2 = 8, outside the window. Example: cycle 6 has p=1, q=00, r0=r1=1. At cycle 7 the state is p=1, q0=q1=1, so g1 fires and g0 would come at cycle 8.
7	SAT	State p=1, q0=q1=1 at cycle 7 gives g0=0 inside the window.

2.1(b): **AG(r1 ⇒ F g1)**

- q1 is held until it is granted. If q1=1 and g1=0, then q1+=1.
- if q1(k)=1, then g1 fires at k or k+1.
  i. If p=1, then g1=q1=1 immediately.
  ii. If p=0 and q0=0, then g1=q1·!q0=1 immediately.
  iii. If p=0 and q0=1, then g0=1 and g1=0. So p+=1, and q1+=q1·!g1=1. At k+1, p=1, so g1=q1=1.

Request at cycle k ≥ 1 with r1(k)=1.

- If q1(k)=0, then q1(k+1)=1, so g1 fires at k+1 or k+2.
- If q1(k)=1, then g1 fires at k or k+1.

So g1 fires within k..k+2, and the property holds for k=1..5 (UNSAT). The three special cases mirror (a):

- k=0 is SAT because the reset cycle discards the request.
- k=6 is SAT with p=0, q=00 at cycle 6, then q0=q1=1 at cycle 7. g0 fires at cycle 7 and g1 would come at cycle 8.
- k=7 is SAT with the same state at cycle 7: q0=q1=1, p=0, so g1=0.

2.2 modified circuit, p(k) = 0 for all k
p is fixed at 0 and its update equation is dropped. Then:

- g0 = q0
- g1 = q1·!q0

q0 can never be high two cycles in a row. With g0=q0, q0+ = r0·!q0 + q0·!q0 = r0·!q0. If q0=1, then q0+=0. This is what stops client 0 from starving client 1. Even with r0 held high, q0 is high at most every other cycle.
Request at cycle k ≥ 1 with r1(k)=1.

- If q1(k)=1:
  - If q0(k)=0, then g1(k)=1 at once.
  - If q0(k)=1, then g1(k)=0, q1 stays 1 (q1+=q1·!g1), and q0(k+1)=0. So g1(k+1)=1.

- If q1(k)=0, then q1(k+1)=1:
  - If q0(k+1)=0, then g1(k+1)=1.
  - If q0(k+1)=1, then q0(k+2)=0 and q1(k+2)=1, so g1(k+2)=1.

So g1 fires within k..k+2, and AG(r1 ⇒ F g1) still holds for k=1..5 (UNSAT).
Worst case. Hold r0=1 and raise r1 at cycle k:

Cycle	r0	r1	q0	q1	g0	g1
k	1	1	0	0	0	0
k+1	1	0	1	1	1	0
k+2	1	0	0	1	0	1

Client 1 waits one extra cycle but is served at k+2.
Window edge cases (as in 1(b)):

- k=0 is SAT because of the reset cycle.
- k=6 is SAT with q=00 at cycle 6, r0=r1=1, then q0=q1=1 at cycle 7, so g1(7)=0.
- k=7 is SAT with the same cycle-7 state.

These are limits of the eight-cycle window, not real starvation.

Finally,

Property	**k=1..5**	**Verdict**
(c) ¬EF(g0·g1)	UNSAT at every k	holds
(a) AG(r0 ⇒ F g0)	UNSAT	holds
(b) AG(r1 ⇒ F g1)	UNSAT	holds
Part 2: p≡0, AG(r1 ⇒ F g1)	UNSAT	still holds

Implementation:
1. arbiter_sat.py:

```
#!/usr/bin/env python3
"""EE709 Assignment 1, Part 2: bounded SAT checks of the arbiter (PySAT +
MiniSat).

Run:  python3 arbiter_sat.py | tee sat_run.log
Also writes DIMACS files (cnf/*.cnf) so you can run `minisat file.cnf` directly.
Assumed equations (overbars restored):
  q0+ = !reset.(r0.!q0 + q0.!g0)    q1+ = !reset.(r1.!q1 + q1.!g1)
  p+  = !reset.(p xor (g0+g1))
  g0  = p ? q0.!q1 : q0         g1  = p ? q1 : q1.!q0
"""
import os, sys
from pysat.formula import IDPool, CNF
from pysat.solvers import Minisat22

N = 8  # window: cycles k = 0..7

class Enc:
    def __init__(self, p_zero=False):
        self.pool, self.cnf = IDPool(), CNF()
        v = self.v = lambda name, k: self.pool.id((name, k))
        self.T = self.pool.id('TRUE'); self.cnf.append([self.T])
        for k in range(N):
            for n in ('r0', 'r1', 'q0', 'q1', 'p'):
                v(n, k)
        self.g = {}
```

```
        for k in range(N):
            q0, q1, p, r0, r1 = (v(n, k) for n in ('q0', 'q1', 'p', 'r0', 'r1'))
            # g0 = p ? q0.!q1 : q0 ; g1 = p ? q1 : q1.!q0
            g0 = self.MUX(p, self.AND(q0, -q1), q0)
            g1 = self.MUX(p, q1, self.AND(q1, -q0))
            self.g[k] = (g0, g1)
            if p_zero:
                self.cnf.append([-p])               # modified circuit: p(k)=0
for all k
            if k + 1 < N:
                if k == 0:                           # reset(0)=1 -> next state
all 0
                    for n in ('q0', 'q1', 'p'):
                        self.cnf.append([-v(n, 1)])
                else:                                # reset(k)=0
                    self.EQ(v('q0', k+1), self.OR(self.AND(r0, -q0),
self.AND(q0, -g0)))
                    self.EQ(v('q1', k+1), self.OR(self.AND(r1, -q1),
self.AND(q1, -g1)))
                    if not p_zero:                   # in the modified circuit p
is fixed, no update
                        self.EQ(v('p', k+1), self.XOR(p, self.OR(g0, g1)))
    # --- Tseitin gates ---
    def new(self): return self.pool.id(('t', self.pool.top + 1))
    def AND(self, a, b):
        z = self.new(); self.cnf.extend([[-z, a], [-z, b], [z, -a, -b]]); return
z
    def OR(self, a, b):
        z = self.new(); self.cnf.extend([[z, -a], [z, -b], [-z, a, b]]); return
z
    def XOR(self, a, b):
        z = self.new(); self.cnf.extend([[-z, a, b], [-z, -a, -b], [z, -a, b],
[z, a, -b]]); return z
    def MUX(self, s, a, b):                          # s ? a : b
        z = self.new(); self.cnf.extend([[-s, -a, z], [-s, a, -z], [s, -b, z],
[s, b, -z]]); return z
    def EQ(self, a, b): self.cnf.extend([[-a, b], [a, -b]])

def run(enc, name, assumptions):
    s = Minisat22(bootstrap_with=enc.cnf.clauses)
    sat = s.solve(assumptions=assumptions)
    m = set(s.get_model()) if sat else None
    s.delete()
    return sat, m

def trace(enc, m):
    b = lambda x: int(x in m)
    print("      k : r0 r1 | p q0 q1 | g0 g1")
    for k in range(N):
        v, g = enc.v, enc.g[k]
```

```
        print(f"      {k} :  {b(v('r0',k))}  {b(v('r1',k))} | {b(v('p',k))}
{b(v('q0',k))}  {b(v('q1',k))} |  {b(g[0])}  {b(g[1])}")

def dump(enc, fname):
    os.makedirs('cnf', exist_ok=True); enc.cnf.to_file('cnf/' + fname)

def check_response(enc, c, ks, label, show=True):
    """Violation of r_c(k) => F g_c : r_c(k) & !g_c(j) for all j in k..N-1."""
    print(f"  {label}")
    for k in ks:
        a = [enc.v(f'r{c}', k)] + [-enc.g[j][c] for j in range(k, N)]
        sat, m = run(enc, label, a)
        print(f"    k={k}: {'SAT  (violation found)' if sat else 'UNSAT (no
violation)'}")
        if sat and show:
            trace(enc, m); show = False

if __name__ == '__main__':
    E = Enc()
    dump(E, 'base_unrolled.cnf')
    print("=== Property (c): not EF(g0.g1), cycles 0..7 ===")
    for k in range(N):
        sat, m = run(E, 'c', [E.g[k][0], E.g[k][1]])
        print(f"  k={k}: {'SAT (g0.g1 reachable)' if sat else 'UNSAT'}")
    for c, tag in ((0, 'a'), (1, 'b')):
        print(f"\n=== Property ({tag}): AG(r{c} => F g{c}) ===")
        check_response(E, c, [0], f"({tag}) k=0 (reset cycle, request is
discarded)")
        check_response(E, c, range(1, 6), f"({tag}) k=1..5 (grant due by k+2 <=
7)")
        check_response(E, c, range(6, 8), f"({tag}) k=6..7 (grant may fall
outside window)")
    print("\n=== Part 2: p(k)=0 for all k, AG(r1 => F g1) ===")
    E0 = Enc(p_zero=True); dump(E0, 'p_zero_unrolled.cnf')
    check_response(E0, 1, range(1, 6), "(2) k=1..5")
```

2. expected log run:

```
=== Property (c): not EF(g0.g1), cycles 0..7 ===
  k=0: UNSAT
  k=1: UNSAT
  k=2: UNSAT
  k=3: UNSAT
  k=4: UNSAT
  k=5: UNSAT
  k=6: UNSAT
  k=7: UNSAT

=== Property (a): AG(r0 => F g0) ===
  (a) k=0 (reset cycle, request is discarded)
    k=0: SAT  (violation found)
      k : r0 r1 | p q0 q1 | g0 g1
```

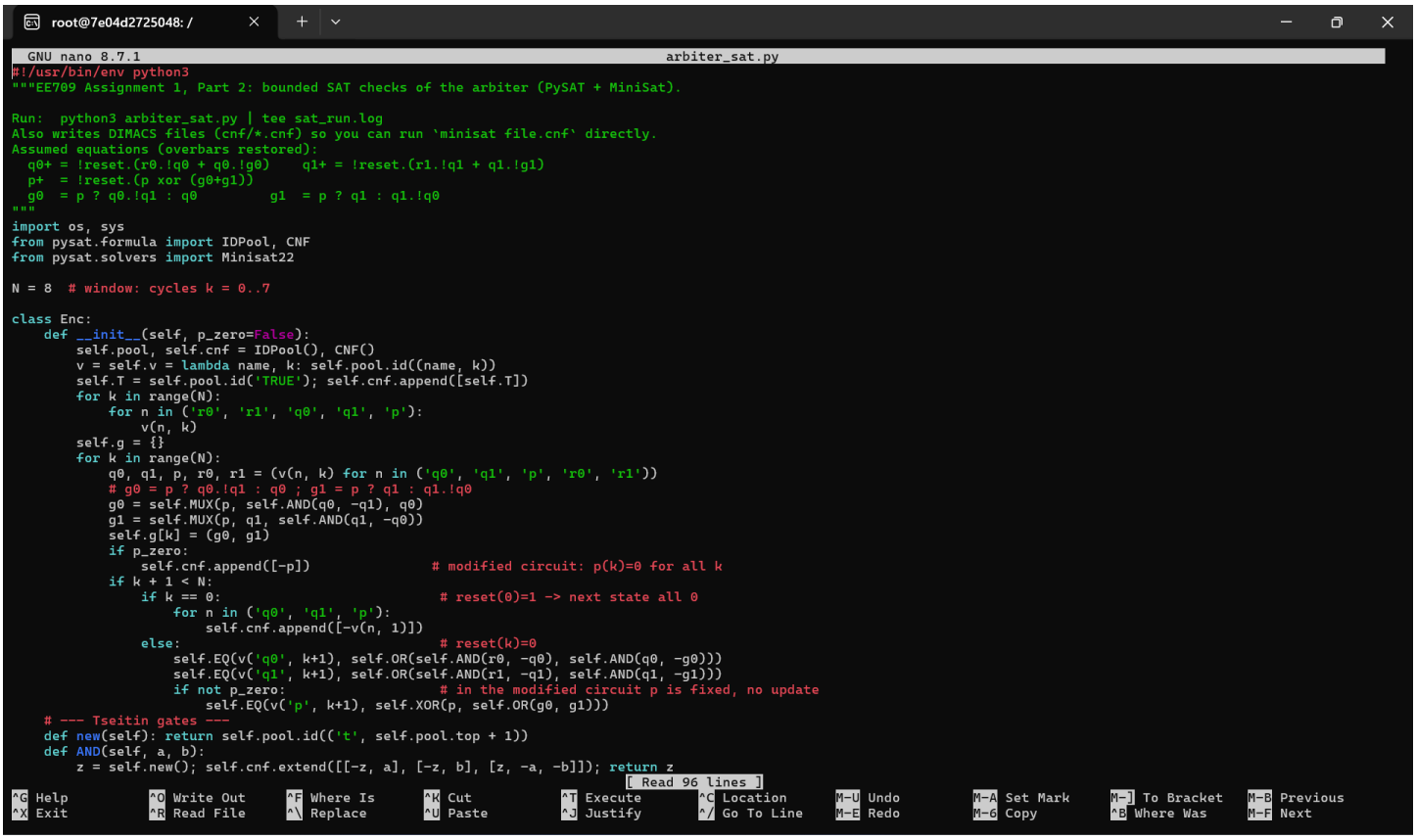
```
      0 :  1  0 | 0  0  0 |  0  0
      1 :  0  0 | 0  0  0 |  0  0
      2 :  0  0 | 0  0  0 |  0  0
      3 :  0  0 | 0  0  0 |  0  0
      4 :  0  0 | 0  0  0 |  0  0
      5 :  0  0 | 0  0  0 |  0  0
      6 :  0  0 | 0  0  0 |  0  0
      7 :  0  0 | 0  0  0 |  0  0
  (a) k=1..5 (grant due by k+2 <= 7)
    k=1: UNSAT (no violation)
    k=2: UNSAT (no violation)
    k=3: UNSAT (no violation)
    k=4: UNSAT (no violation)
    k=5: UNSAT (no violation)
  (a) k=6..7 (grant may fall outside window)
    k=6: SAT  (violation found)
      k : r0 r1 | p q0 q1 | g0 g1
      0 :  0  0 | 0  0  0 |  0  0
      1 :  0  0 | 0  0  0 |  0  0
      2 :  0  0 | 0  0  0 |  0  0
      3 :  0  0 | 0  0  0 |  0  0
      4 :  0  1 | 0  0  0 |  0  0
      5 :  0  0 | 0  0  1 |  0  1
      6 :  1  1 | 1  0  0 |  0  0
      7 :  0  0 | 1  1  1 |  0  1
    k=7: SAT  (violation found)

=== Property (b): AG(r1 => F g1) ===
  (b) k=0 (reset cycle, request is discarded)
    k=0: SAT  (violation found)
      k : r0 r1 | p q0 q1 | g0 g1
      0 :  0  1 | 0  0  0 |  0  0
      1 :  0  0 | 0  0  0 |  0  0
      2 :  0  0 | 0  0  0 |  0  0
      3 :  0  0 | 0  0  0 |  0  0
      4 :  0  0 | 0  0  0 |  0  0
      5 :  0  0 | 0  0  0 |  0  0
      6 :  0  0 | 0  0  0 |  0  0
      7 :  0  0 | 0  0  0 |  0  0
  (b) k=1..5 (grant due by k+2 <= 7)
    k=1: UNSAT (no violation)
    k=2: UNSAT (no violation)
    k=3: UNSAT (no violation)
    k=4: UNSAT (no violation)
    k=5: UNSAT (no violation)
  (b) k=6..7 (grant may fall outside window)
    k=6: SAT  (violation found)
      k : r0 r1 | p q0 q1 | g0 g1
      0 :  0  0 | 0  0  0 |  0  0
      1 :  0  0 | 0  0  0 |  0  0
      2 :  0  0 | 0  0  0 |  0  0
```

```
       3 :  0  0 | 0  0  0 |  0  0
       4 :  0  0 | 0  0  0 |  0  0
       5 :  0  0 | 0  0  0 |  0  0
       6 :  1  1 | 0  0  0 |  0  0
       7 :  0  0 | 0  1  1 |  1  0
    k=7: SAT  (violation found)

=== Part 2: p(k)=0 for all k, AG(r1 => F g1) ===
  (2) k=1..5
    k=1: UNSAT (no violation)
    k=2: UNSAT (no violation)
    k=3: UNSAT (no violation)
    k=4: UNSAT (no violation)
    k=5: UNSAT (no violation)
```

3. insert code in nano:



4. Output terminal:

```
(sat-env) root@7e04d2725048:/# nano arbiter_sat.py
(sat-env) root@7e04d2725048:/#  cd /
python3 arbiter_sat.py | tee sat_run.log
minisat cnf/base_unrolled.cnf | tee minisat_base.log
minisat cnf/p_zero_unrolled.cnf | tee minisat_pzero.log
=== Property (c): not EF(g0.g1), cycles 0..7 ===
  k=0: UNSAT
  k=1: UNSAT
  k=2: UNSAT
  k=3: UNSAT
  k=4: UNSAT
  k=5: UNSAT
  k=6: UNSAT
  k=7: UNSAT
```

Figure: Property (c), UNSAT for k = 0..7. The first lines show the commands used

```
=== Property (a): AG(r0 => F g0) ===
  (a) k=0 (reset cycle, request is discarded)
    k=0: SAT  (violation found)
      k : r0 r1 | p q0 q1 | g0 g1
      0 :  1  0 | 0  0  0 |  0  0
      1 :  0  0 | 0  0  0 |  0  0
      2 :  0  0 | 0  0  0 |  0  0
      3 :  0  0 | 0  0  0 |  0  0
      4 :  0  0 | 0  0  0 |  0  0
      5 :  0  0 | 0  0  0 |  0  0
      6 :  0  0 | 0  0  0 |  0  0
      7 :  0  0 | 0  0  0 |  0  0
  (a) k=1..5 (grant due by k+2 <= 7)
    k=1: UNSAT (no violation)
    k=2: UNSAT (no violation)
    k=3: UNSAT (no violation)
    k=4: UNSAT (no violation)
    k=5: UNSAT (no violation)
  (a) k=6..7 (grant may fall outside window)
    k=6: SAT  (violation found)
      k : r0 r1 | p q0 q1 | g0 g1
      0 :  0  0 | 0  0  0 |  0  0
      1 :  0  0 | 0  0  0 |  0  0
      2 :  1  0 | 0  0  0 |  0  0
      3 :  0  0 | 0  1  0 |  1  0
      4 :  0  0 | 1  0  0 |  0  0
      5 :  0  0 | 1  0  0 |  0  0
      6 :  1  1 | 1  0  0 |  0  0
      7 :  0  0 | 1  1  1 |  0  1
    k=7: SAT  (violation found)
```

Figure: Property (a): UNSAT for k = 1..5; SAT only at k = 0 and k = 6, 7 (with traces).

```
=== Property (b): AG(r1 => F g1) ===
  (b) k=0 (reset cycle, request is discarded)
    k=0: SAT  (violation found)
      k : r0 r1 | p q0 q1 | g0 g1
      0 :  0  1 | 0  0  0 |  0  0
      1 :  0  0 | 0  0  0 |  0  0
      2 :  0  0 | 0  0  0 |  0  0
      3 :  0  0 | 0  0  0 |  0  0
      4 :  0  0 | 0  0  0 |  0  0
      5 :  0  0 | 0  0  0 |  0  0
      6 :  0  0 | 0  0  0 |  0  0
      7 :  0  0 | 0  0  0 |  0  0
  (b) k=1..5 (grant due by k+2 <= 7)
    k=1: UNSAT (no violation)
    k=2: UNSAT (no violation)
    k=3: UNSAT (no violation)
    k=4: UNSAT (no violation)
    k=5: UNSAT (no violation)
  (b) k=6..7 (grant may fall outside window)
    k=6: SAT  (violation found)
      k : r0 r1 | p q0 q1 | g0 g1
      0 :  0  0 | 0  0  0 |  0  0
      1 :  0  0 | 0  0  0 |  0  0
      2 :  0  0 | 0  0  0 |  0  0
      3 :  0  0 | 0  0  0 |  0  0
      4 :  0  0 | 0  0  0 |  0  0
      5 :  0  0 | 0  0  0 |  0  0
      6 :  1  1 | 0  0  0 |  0  0
      7 :  0  0 | 0  1  1 |  1  0
    k=7: SAT  (violation found)
```

Figure: Property (b): UNSAT for k = 1..5; SAT only at k = 0 and k = 6, 7 (with traces).

```
=== Part 2: p(k)=0 for all k, AG(r1 => F g1) ===
  (2) k=1..5
    k=1: UNSAT (no violation)
    k=2: UNSAT (no violation)
    k=3: UNSAT (no violation)
    k=4: UNSAT (no violation)
    k=5: UNSAT (no violation)
WARNING: for repeatability, setting FPU to use double precision
============================[ Problem Statistics ]=============================
|                                                                             |
|  Number of variables:          121                                          |
|  Number of clauses:            259                                          |
|  Parse time:                  0.00 s                                        |
|  Eliminated clauses:          0.00 Mb                                       |
|  Simplification time:         0.00 s                                        |
|                                                                             |
============================[ Search Statistics ]==============================
| Conflicts |          ORIGINAL         |          LEARNT          | Progress |
|           |    Vars  Clauses Literals |    Limit  Clauses Lit/Cl |          |
===============================================================================
===============================================================================
restarts              : 1
conflicts             : 0              (-nan /sec)
decisions             : 1              (0.00 % random) (inf /sec)
propagations          : 13             (inf /sec)
conflict literals     : 0              (-nan % deleted)
Memory used           : 12.00 MB
CPU time              : 0 s

SATISFIABLE
```

Figure: Part 2 (p ≡ 0): UNSAT for k = 1..5. Below it, MiniSat on cnf/base_unrolled.cnf: 121 variables, 259 clauses, SATISFIABLE.

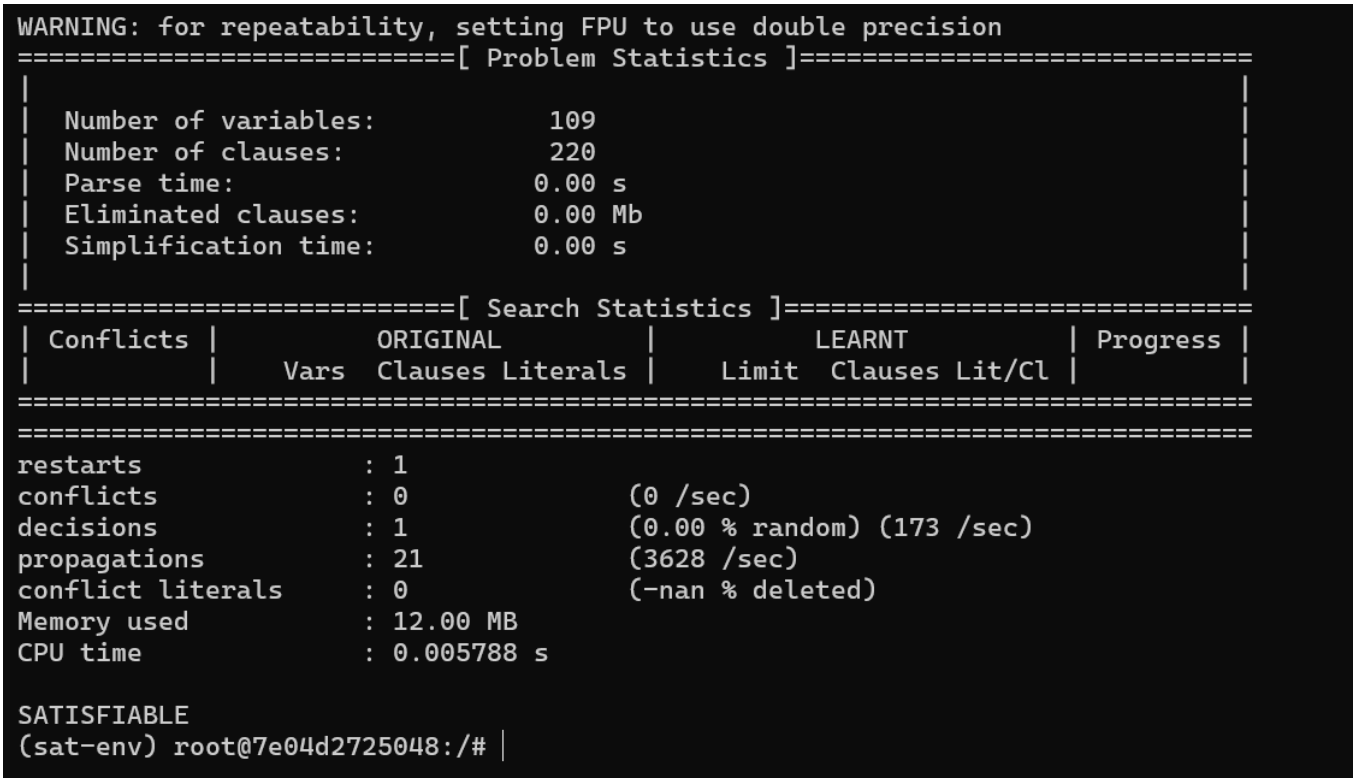


Figure: MiniSat on cnf/p_zero_unrolled.cnf: 109 variables, 220 clauses, SATISFIABLE.

MiniSat: They solve the unrolled circuits with free inputs and no property assumptions, so SATISFIABLE only means the encodings are consistent (the circuits can run for eight cycles). The property verdicts are the UNSAT/SAT lines printed by arbiter_sat.py (PySAT, MiniSat22)

Requirements and commands used:

1. Requirements:

Requirement	Used for	Notes / installation
Windows with WSL2 (Ubuntu) and Docker Desktop (WSL integration enabled)	host environment	Ubuntu terminal runs the docker command
Ubuntu-based Docker container (peaceful_shannon), root shell	runs all programs	docker start -ai peaceful_shannon
Python 3, pip, venv	runs both scripts	apt-get install -y python3 python3-pip python3-venv
PySAT (pip package python-sat)	arbiter_sat.py: CNF, Minisat22 solver	inside a virtual environment: pip install python-sat
MiniSat command-line solver	solves the dumped DIMACS files	apt-get install -y minisat
nano (any editor works)	creating the source files	apt-get install -y nano
Nothing else	bdd_reach.py uses only the Python standard library (itertools)	no extensions, plug-ins or BDD library needed

2. Commands used.

i. Step 1 (start the container, from the WSL Ubuntu terminal):
   - `docker start -ai peaceful_shannon`

ii. Step 2 (one-time setup, inside the container as root):
   - `export DEBIAN_FRONTEND=noninteractive`
   - `apt-get update`
   - `apt-get install -y nano python3 python3-pip python3-venv minisat`
   - `python3 -m venv /root/sat-env`
   - `source /root/sat-env/bin/activate`
   - `pip install python-sat`
   - `python3 -c "import pysat; print(pysat.__version__)"   # check`

iii. Step 3 (Question 1: create the file with nano, paste bdd_reach.py, then run):
   - `nano bdd_reach.py`
   - `python3 bdd_reach.py | tee bdd_run.log`

iv. Step 4 (Question 2: create the file with nano, paste arbiter_sat.py:, then run the SAT queries and the MiniSat checks):
   - `nano arbiter_sat.py`
   - `python3 arbiter_sat.py | tee sat_run.log`
   - `minisat cnf/base_unrolled.cnf   | tee minisat_base.log`
   - `minisat cnf/p_zero_unrolled.cnf | tee minisat_pzero.log`

3. Outputs: bdd_run.log, sat_run.log, minisat_base.log, minisat_pzero.log, and the folder cnf/ with the two DIMACS files. Each script runs in well under a second. The virtual environment must be active (step 2) whenever arbiter_sat.py is run.